3.modelling

Joao Lopes

2026-03-25

Contents

1. BUILD A MODEL **3**
1.1. LOOK AT DATA . . . 3
1.2. FIT A MODEL . . . 4
1.3. USE MODEL TO PREDICT . . . 4

2. BUILD A MODEL USING WORKFLOW **6**
2.1. LOOK AT DATA . . . 6
2.2. SPLIT DATA . . . 7
2.3. CREATE MODEL . . . 7
2.4. CREATE RECIPE . . . 7
2.5. CREATE WORKFLOW . . . 7
2.6. FIT A MODEL . . . 8
2.7. EVALUATE MODEL . . . 8

3. BUILD VARIOUS MODELS USING WORKFLOW **10**
3.1. LOOK AT DATA . . . 10
3.2. SPLIT DATA . . . 10
3.3. CREATE WORKFLOW AND FIT MODEL 1 . . . 11
3.4. CREATE WORKFLOW AND FIT MODEL 2 . . . 12
3.5. EVALUATE LAST MODEL . . . 14

1. BUILD A MODEL

[from https://www.tidymodels.org/start/models/]

Problem: Predict the price of a diamond.

Formula: price ~ carat + cut

Model: lm()

```
library("skimr")        #function skim() for descriptive statistics
library("GGally")       #function ggpairs() for data visualization
library("hexbin")       #function geom_hex() for data visualization
library("dotwhisker")   #function dwplot() for model coefficients visualization
library("performance") #function check_model() for linear model assumptions
library("tidymodels")   #collection of packages for modelling
library("tidyverse")    #collection of packages for data analysis

tidyverse_conflicts()
tidymodels_conflicts()
```

1.1. LOOK AT DATA

```
#loading data and making changes
set.seed(123)
lev_cut = c("Fair", "Good", "Very Good", "Premium", "Ideal")
diamonds_data <- diamonds |>
  mutate(
    log_price = log(price),
    log_carat = log(carat),
    cut = fct(as.character(cut), levels = lev_cut),
    .keep = "used"
  ) |>
  slice_sample(n = 1000)

#inspect data
glimpse(diamonds_data)

#descriptive statistics
skim(diamonds_data)
```

```
#data visualization 1
diamonds_data |>
  select(-c(carat, price)) |>
  ggpairs(progress = FALSE)
```

```
#data visualization 2
ggplot(diamonds_data, aes(x = log_carat, y = log_price)) +
  geom_hex() +
  facet_wrap(~cut, ncol = 1)
```

```
#data visualization 3
ggplot(diamonds_data, aes(x = log_carat, y = log_price, color = cut)) +
  geom_smooth(method = "lm", formula = "y ~ x")
```

1.2. FIT A MODEL

```r
#set functional form
?linear_reg

#set engine
?set_engine

#perform fit
?fit.model_spec()
```

```r
#fit model
t1 <- Sys.time()
lm_fit <- linear_reg() |>
  set_engine("lm") |>
  fit(formula = log_price ~ log_carat + cut, data = diamonds_data)
Sys.time() - t1

#analyse model (classic regression table)
tidy(lm_fit)
glance(lm_fit)

#predict using same data used for the fitting
diamonds_data_aug <- augment(lm_fit, new_data = diamonds_data)
diamonds_data_aug

#evaluate model using R-square (over evaluation)
diamonds_data_aug |>
  rsq(truth = log_price, estimate = .pred)

#evaluate model using RMSE (over evaluation)
diamonds_data_aug |>
  yardstick::rmse(truth = log_price, estimate = .pred)

#plot results
tidy(lm_fit) |>
  dwplot(
    dot_args = list(size = 2, color = "black"),
    whisker_args = list(color = "black"),
    vline = geom_vline(xintercept = 0, colour = "grey50", linetype = 2))
```

```r
#check MLR assumptions
check_model(lm_fit$fit)
```

1.3. USE MODEL TO PREDICT

```r
#create new points
new_points <- expand.grid(
  log_carat = log(2),
  cut = c("Fair", "Very Good", "Ideal")
  )
new_points

#get prediction
```

```r
mean_pred <- predict(lm_fit, new_data = new_points)
mean_pred

#get confidence interval
conf_int_pred <- predict(lm_fit, new_data = new_points, type = "conf_int")
conf_int_pred

#plot points
new_points |>
  bind_cols(mean_pred, conf_int_pred) |>
  ggplot(aes(x = cut)) +
  geom_point(aes(y = exp(.pred))) +
  geom_errorbar(
    aes(ymin = exp(.pred_lower), ymax = exp(.pred_upper)),
    width = .2
  ) +
  labs(x = "diamond cut", y = "diamond price")
```

---

2. BUILD A MODEL USING WORKFLOW

[from https://www.tidymodels.org/start/recipes/]

Problem: Predict if a flight is going to arrive late.

Formula: arr_delay ~ air_time + dep_time + distance + carrier + dest + origin + date_dow + date_month + date_USholiday

Model: glm(family = binomial(link = "logit"))

```
library("nycflights13") #collection of datasets
library("skimr")        #function skim() for descriptive statistics
library("GGally")       #function ggpairs() for data visualization
library("tidymodels")   #collection of packages for modelling
library("tidyverse")    #collection of packages for data analysis
```

2.1. LOOK AT DATA

```
#loading data and making changes
flight_data <-
  flights |>
  mutate(
    #create variable on arrival delay
    arr_delay = ifelse(arr_delay >= 30, "late", "on_time"),
    arr_delay = fct(arr_delay, levels = c("late", "on_time")),
    #create variable date
    date = lubridate::as_date(time_hour)
  ) |>
  #select the variables that are going to be used
  select(dep_time, flight, origin, dest, air_time, distance,
         carrier, date, arr_delay, time_hour) |>
  #exclude missing data
  drop_na() |>
  #create factor variables
  mutate(across(where(is.character), as.factor))

#looking at data
glimpse(flight_data)

#descriptive statistcs
skim(flight_data)
```

```
#data visualization
set.seed(123)
flight_data |>
  slice_sample(n=1000) |>
  select(-c(
    time_hour, flight, #ID of the flights
    dest, carrier,     #too many levels
    dep_time, date     #time and data
    )
  ) |>
  select(arr_delay, where(is.factor), where(is.numeric)) |>
  ggpairs(progress = FALSE)
```

2.2. SPLIT DATA

```
#create random data split
set.seed(222)
data_split <- initial_split(flight_data, strata = arr_delay, prop = 3/4)

train_data <- training(data_split) #extract training data (n = 245510)
test_data  <- testing(data_split)  #extract testing data   (n = 81836)

#look for unbalance in binary dependent variable (training data)
train_data |>
  count(arr_delay) |>
  mutate(prop = n/sum(n))

#look for unbalance in binary dependent variable (testing data)
test_data |>
  count(arr_delay) |>
  mutate(prop = n/sum(n))
```

2.3. CREATE MODEL

```
#create model specification
lr_mod <-
  logistic_reg() |>
  set_engine("glm")
```

2.4. CREATE RECIPE

```
#create recipe
flights_rec <-
  #new recipe
  recipe(formula = arr_delay ~ ., data = train_data) |>
  #add role to recipe
  update_role(flight, time_hour, new_role = "ID") |>
  #add date features
  step_date(date, features = c("dow", "month")) |>
  step_holiday(date, holidays = timeDate::listHolidays("US")) |>
  step_rm(date) |>
  #convert categorical variables to dummies
  step_dummy(all_nominal_predictors()) |>
  #remove columns with zero variance
  step_zv(all_predictors())
```

2.5. CREATE WORKFLOW

```
#create workflow
flights_wflow <-
  #new workflow
  workflow() |>
  #add model
  add_model(lr_mod) |>
  #add recipe
```

```r
  add_recipe(flights_rec)
flights_wflow
```

2.6. FIT A MODEL

```r
#fit the model using training data
t1 <- Sys.time()
glm_fit <-
  flights_wflow |>
  fit(data = train_data)
Sys.time() - t1

#analyse model (classic regression table)
tidy(glm_fit)
glance(glm_fit)
```

2.7. EVALUATE MODEL

```r
#calculate predictions using testing data
flights_aug <-
  augment(glm_fit, new_data = test_data)
flights_aug |>
  select(arr_delay, time_hour, flight, .pred_class, .pred_on_time) |>
  print(n = 20)

#evaluate model using confusion matrix
lr_cm <- flights_aug |>
  conf_mat(truth = arr_delay, estimate = .pred_class)
lr_cm

summary(lr_cm)

#evaluate model using ROC curve
flights_aug |>
  roc_curve(truth = arr_delay, .pred_late) |>
  autoplot()

#evaluate model using ROC area
flights_aug |>
  roc_auc(truth = arr_delay, .pred_late)

#extract results
glm_fit |>
  extract_fit_parsnip() |>
  tidy() |>
  filter(!term %in% c("(Intercept)")) |>
  mutate(
    importance = abs(statistic),
    term = fct_reorder(term, importance)
  ) |>
  arrange(desc(importance)) |>
  slice_head(n = 20) |>
```

```r
ggplot(aes(x = importance, y = term)) +
geom_col()
```

---

3. BUILD VARIOUS MODELS USING WORKFLOW

[from https://www.tidymodels.org/start/case-study/]

Problem: Predict if a booking has children.

Formula: children ~ .

Model 1: glmnet(family = "binomial")

Model 2: ranger(classification = TRUE)

```
library("skimr")       #function skim() for descriptive statistics
library("GGally")      #function ggpairs() for data visualization
library("glmnet")      #function glmnet() to fit GLMs with penalized mL
library("ranger")      #function ranger() to random forest models
library("vip")         #function vip() to plot "vars importance" for ML models
library("tidymodels") #collection of packages for modelling
library("tidyverse")  #collection of packages for data analysis
```

3.1. LOOK AT DATA

```
#loading data and making changes
hotels <-
  read_csv("https://tidymodels.org/start/case-study/hotels.csv") |>
  mutate(across(where(is.character), as.factor))

#inspect data
glimpse(hotels)

#descriptive statistics
skim(hotels)
```

```
#data visualization
set.seed(123)
hotels |>
  slice_sample(n=1000) |>
  select(-c(
    required_car_parking_spaces, customer_type,  #too unbalanced
    meal:assigned_room_type,                     #too many levels
    stays_in_weekend_nights, is_repeated_guest:days_in_waiting_list,
      total_of_special_requests,                 #too many 0s
    arrival_date                                 #data
    )
  ) |>
  select(children, where(is.factor), where(is.numeric)) |>
  ggpairs(progress = FALSE)
```

3.2. SPLIT DATA

```
#create random data split
set.seed(123)
splits      <- initial_split(hotels, strata = children, prop = 0.75)

hotel_other <- training(splits) #extract training data (n = 37500)
hotel_test  <- testing(splits)  #extract testing data  (n = 12500)
```

```r
#look for unbalance in binary dependent variable (training data)
hotel_other |>
  count(children) |>
  mutate(prop = n/sum(n))

#look for unbalance in binary dependent variable (testing data)
hotel_test  |>
  count(children) |>
  mutate(prop = n/sum(n))

#create random split of training data (n_training = 30000, n_tuning = 7500)
set.seed(234)
val_set <- validation_split(hotel_other, strata = children, prop = 0.80)
```

3.3. CREATE WORKFLOW AND FIT MODEL 1

```r
#create model specification
?logistic_reg
```

```r
lr_mod <-
  logistic_reg(
    penalty = tune(), #total amount of regularization
    mixture = 1       #choose ridge (0), lasso (1), or elastic models (0 to 1)
  ) |>
  set_engine("glmnet")

#create recipe
holidays <- c("AllSouls", "AshWednesday", "ChristmasEve", "Easter",
              "ChristmasDay", "GoodFriday", "NewYearsDay", "PalmSunday")
lr_recipe <-
  #new recipe
  recipe(children ~ ., data = hotel_other) |>
  #add date features
  step_date(arrival_date) |>
  step_holiday(arrival_date, holidays = holidays) |>
  step_rm(arrival_date) |>
  #convert categorical variables to dummies
  step_dummy(all_nominal_predictors()) |>
  #remove columns with zero variance
  step_zv(all_predictors()) |>
  #normalize numeric data
  step_normalize(all_predictors())

#create workflow
lr_workflow <-
  #new workflow
  workflow() |>
  #add model
  add_model(lr_mod) |>
  #add recipe
  add_recipe(lr_recipe)
lr_workflow
```

```r
#fit model using training data and evaluate using tuning data
t1 <- Sys.time()
lr_res <-
  lr_workflow |>
  tune_grid(
    resamples = val_set,
    grid = 20,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc)
  )
Sys.time() - t1

#plot tuning metrics
autoplot(lr_res)

#select best tuning parameter
lr_best <-
  lr_res |>
  select_best(metric = "roc_auc")
lr_best

#collect predictions using best tuned model
lr_pred <- lr_res |>
  collect_predictions(parameters = lr_best)

#evaluate model using ROC curve
lr_auc <-
  lr_pred |>
  roc_curve(children, .pred_children) |>
  mutate(model = "Logistic Regression")
autoplot(lr_auc)

#evaluate model using ROC area
lr_pred |>
  roc_auc(children, .pred_children)
```

3.4. CREATE WORKFLOW AND FIT MODEL 2

```r
#number of cores to parallelize computation
cores <- parallel::detectCores()
```

```r
#create model specification
?rand_forest
```

```r
rf_mod <-
  rand_forest(
    mtry = tune(),  #number of predictors for each split
    min_n = tune(), #minimum number of data points in a node to stop the split
    trees = 1000    #number of trees contained in the ensemble
  ) |>
  set_engine("ranger", num.threads = cores) |>
  set_mode("classification")

#create recipe
```

```r
holidays <- c("AllSouls", "AshWednesday", "ChristmasEve", "Easter",
              "ChristmasDay", "GoodFriday", "NewYearsDay", "PalmSunday")
rf_recipe <-
  #new recipe
  recipe(children ~ ., data = hotel_other) |>
  #add date features
  step_date(arrival_date) |>
  step_holiday(arrival_date, holidays = holidays) |>
  step_rm(arrival_date)

#create workflow
rf_workflow <-
  #new workflow
  workflow() |>
  #add model
  add_model(rf_mod) |>
  #add recipe
  add_recipe(rf_recipe)
rf_workflow

#fit model using training data and evaluate using tuning data
set.seed(345)
t1 <- Sys.time()
rf_res <-
  rf_workflow |>
  tune_grid(
    resamples = val_set,
    grid = 10,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc)
  )
Sys.time() - t1

#plot tuning metrics
autoplot(rf_res)

#select best parameters
rf_best <-
  rf_res |>
  select_best(metric = "roc_auc")
rf_best

#collect predictions using best tuned model
rf_pred <-
  rf_res |>
  collect_predictions(parameters = rf_best)

#evaluate model using ROC curve
rf_auc <-
  rf_pred |>
  roc_curve(children, .pred_children) |>
  mutate(model = "Random Forest")
autoplot(rf_auc)
```

```r
#evaluate model using ROC area
rf_pred |> roc_auc(children, .pred_children)
```

3.5. EVALUATE LAST MODEL

```r
#compare best models
bind_rows(rf_auc, lr_auc) |>
  ggplot(aes(x = 1 - specificity, y = sensitivity, col = model)) +
  geom_path(lwd = 1.0, alpha = 0.8) +
  geom_abline(lty = 3) +
  coord_equal() +
  theme_bw() +
  theme(legend.position = "bottom")

#build last model
last_rf_mod <-
  rand_forest(
    mtry = rf_best$mtry,
    min_n = rf_best$min_n,
    trees = 1000
  ) |>
  set_engine("ranger", num.threads = cores, importance = "impurity") |>
  set_mode("classification")

#create last workflow
last_rf_workflow <-
  #random forest workflow
  rf_workflow |>
  #update with tuned model
  update_model(last_rf_mod)

#fit last model using testing data
set.seed(345)
t1 <- Sys.time()
last_rf_fit <-
  last_rf_workflow |>
  last_fit(split = splits)
Sys.time() - t1

#collect predictions using testing data
last_rf_pred <- last_rf_fit |>
  collect_predictions()

#evaluate model using confusion matrix
last_rf_pred |>
  conf_mat(truth = children, estimate = .pred_class)

#evaluate model using ROC curve
last_rf_pred |>
  roc_curve(children, .pred_children) |>
  autoplot()
```

```r
#evaluate model using ROC area
last_rf_pred |>
  roc_auc(children, .pred_children)

#extract results
last_rf_fit |>
  extract_fit_parsnip() |>
  vip(num_features = 20)
```